

4주차 세션

📅 날짜	@2026년 2월 5일
🌟 상태	완료

1) TRANSFORMER

- Sequential이 중요한 이유
 - 자연어는 단어의 순서에 따라 의미랑 구조가 달라짐.

⇒ 모델은 단어 간 순서 정보를 반드시 고려해야 함.
- n-gram
 - 순서,빈도 기반의 통계적 언어 모델
 - 인간이 봤을 때에는 유사하다는걸 알고있지만 n-gram이 유사하다는 것을 인식하지 못함.

⇒ 희소성 문제가 생김 (보지 못한 n-gram 조합은 처리가 불가능함)
- NNLM
 - 단어를 연속공간의 벡터로 표현 → 임베딩
 - 신경망을 통해 다음 단어의 확률 분포 예측
 - 한계 : 고정 길이 문맥만 사용 가능함 → 긴 문맥을 반영하기 어려움.
- RNN
 - 가변 길이 시퀀스 처리 가능
 - 과거 정보를 hidden state 하나에 계속 누적해서 전달
 - 이론적으로는 입력길이의 제한은 없음.
 - 한계 : 자세히 살펴보면 긴 시퀀스에서는 그래디언트가 매우 작아지거나 커지는 문제가 발생함. → 기울기 소실/폭주 문제
 - 오래된 정보 학습 어려움
- LSTM & GRU
 - 장기 의존성 해결하기 위하여 등장함.
 - 게이트 구조로 정보 흐름 제어

- 한계 : 병렬화 어려움.
- 모든 정보를 하나의 상태에 요약 → 결국 근본적인 장기 의존성 한계 존재.=

2) SEQ2SEQ

- 인코더가 입력 시퀀스를 고정 차원으로 최종 요약 : context vector
- 디코더는 이전 시점 출력을 다음 시점 입력으로 연결함.
 - 테스트 단계에서는 문장이 잘 생성됨.
 - 훈련 단계에서는 이렇게 안함.
 - 틀리게 디코더의 들어가면 누적해서 틀린걸로 잘못되기 때문에
 - 그래서 정답 시퀀스의 이전 단어를 넣어줌.
- 디코더도 문장의 끝을 의미하는 eos가 생성될 때까지 계속해서 훈련
- 입력길이와 출력길이 1대1이 아니어도 처리 가능
- 문제점 : 정보가 병목이 됨. (고정 길이 병목 발생)
- 해결 : attention 사용함.

3) ATTENTION

- 디코더가 각 시점마다 중요한 입력 위치에 집중함.
- attention value가 필요함.
 - attention score 계산 필요함.
- softmax 사용
 - 모든 값이 0~1 사이
 - 가중치로 사용 가능
 - 확률 분포 형태의 attention weight
 - 가중합을 통해서
 - 시점 t마다 context vector 생성
 - seq2seq와의 차이 : 전체에 대한 하나의 벡터를 만드는 것이 아니라, 디코더가 단어 하나를 만들 때마다(각 출력 시점마다) 다른 context vector를 만들.

4) TRANSFORMER

- 어텐션만으로 구성된 모델
- RNN사용하지 않지만 인코더에서 입력 시퀀스 받고 디코더에서 출력 시퀀스 내보내는 구조는 사용
- 각 단어의 위치 정보를 가질 수 있음.
- 단어 입력을 순차적으로 받는게 아님.
- 기존의 RNN은 시점 T의 시간축에 따라서 단어를 하나씩 상태를 업데이트
 - 모든 토큰을 병렬로 처리함 (포지셔널 인코딩이 필요함)
- 트랜스포머의 기본 연산 1: SELF-ATTENTION
 - 한 토큰이 다른 모든 토큰의
- 루트 디케이로 나눠서 스케일 보정
- 소프트맥스로 점수를 확률분포로 바꿈
- 어텐션으로 토큰간의 정보를 섞는다는게 이런 내용이구나
- MULTI-HEAD ATTENTION
 - 왜 헤드를 여러개 쓸까?
 - 언어에서의 관계는 단일하지않다!
 - 관점을 여러개 둔다!
 - 헤드마다 서로다른 가중치 행렬을 가지고 쿼리, 밸류값을 만듦.
 - 병렬 수행을 할 수 있음.
- 디코더에서는 멀티 헤드 어텐션에 마스크를 씌움.
 - 아무제약 없으면 모든 단어가 서로를 보게 됨.
 - 정답을 미리 보는 '치팅'이 되어버려서 마스크를 함.
 - 각 토큰은 자기 자신과 미래만 보게 됨.
- FFNN도 중요함 - POSITION - WISE FEED-FORWARD NETWORKS
 - 토큰같은 경우는 각 위치에 존재하는 표현들
 - 문장 전체를 한번에 처리하는 것이 아니라 각 위치 벡터 하나하나 MLP를 적용함
 - 이미 어텐션이 만들어낸 벡터를 그냥 두지않고 비선형함수를 포함해서 변환을 해서 표현력있게 만든다.
- ADD&NORM

- RESIDUAL CONNECTION
 - 서브레이어의 입력 X 를 출력 레이어에 더해줌.
 - 층이 깊어질수록 정보가 사라져서 어려운것이어서 입력이 흐르도록 해서 기울기 개선함.
 - 기울기가 잘 흘러가면서 잘 고정되어서 끝까지 갈 수 있게끔함.
 - Layernorm
 - 평균,분산 정규화해서 다음층의 입력분포가 안정적으로 되게끔함.
-

2) BERT



- 트랜스포머의 파생 모델
- 문장을 양방향으로 이해하도록 레이어 하나만 파인튜닝해서.
- GPT-1 다음에 등장함
 - 디코더만 사용함 → 미래 토큰을 보지 못하도록 마스킹 처리함 → 단방향
- 아이디어 : 그러면 인코더만 사용하면 문장을 양방향으로 동시에 이해할 수 있찌 ○낱을 까?
- 이진 분류 사전학습
 - MLM
 - NSP
- BERT의 입력
 1. Original text
 2. wordpiece tokenization → 토큰 시퀀스 구성
 3. 스페셜 토큰

- a. CLS
 - i. 시작 (전체 문장을 대표하는 벡터)
 - b. SEP
 - i. 문장 사이사이
- 입력 벡터의 단순 합 → 하나의 벡터
 - 1. Token embedding
 - 2. segment embedding
 - 3. position embedding
- transformer encoder
 - 버트는 인코더의 넣는거 여러번
 - 몇번 반복하느냐? base랑 large까지 제안
- pre-training (2가지)
 - MLM
 - 양방향 정보를 사용하기 위해 단방향인 GPT보다 풍부한 정보를 가지는 언어 모델
 - 입력토큰의 일부를 가리고 가려진 단어를 예측하도록 함.
 - 기존의 LM과 다르게 마스킹되어있는 앞과 뒤를 참고함.
 - 입력 토큰 중에서 약 15퍼센트를 마스크 토큰으로 대체함.
 - 문제점 : FINE-TUNING 과정에는 마스크 토큰이 존재하지 않아서 추가적인 해결이 필요함
 - 해결 : 80%는 마스크로 대체 10%는 랜덤한 단어로 대체, 10%는 원래 단어 그대로 유지함.
 - NSP
 - 문장과 문장 사이를 이해하는게 핵심임
 - 두 문장이 연속된 문장인지 아닌지 이진분류하는 것
- Fine-tuning
 - 출력 레이어 파인튜닝

bert , gpt, bart

구분	BERT	GPT-1	BART
아키텍처	Encoder Only	Decoder Only	Encoder-Decoder
Attention 방향	양방향 (Bidirectional)	단방향 (Left → Right)	양방향 + 생성
학습 목표	MLM, NSP	Autoregressive LM	Denoising Autoencoder
입력 이해	강함	약함	강함
문장 생성	불가능	매우 강함	강함
대표 태스크	NLI, QA, NER	Text Generation	Summarization, Translation
핵심 특징	문맥 이해 특화	생성 특화	이해 + 생성 결합